

Identifying Key Entities in Recipe Data

Named Entity Recognition for Recipe Ingredients using Conditional Random Fields

Prepared by: Nishant Bilagi
Date: June 16, 2026

1. Executive Summary

This report documents the development and evaluation of a Conditional Random Field (CRF) Named Entity Recognition model that classifies tokens in recipe ingredient lists into three categories: ingredient, quantity, and unit. The raw dataset of 285 recipes was cleaned to 280 records, split into 196 training and 84 validation recipes (70:30), and used to train a CRF tagger with class-weighted features to counter the natural dominance of the ingredient label. The resulting model achieved 99.65% token-level accuracy on the validation set, with perfect separation of the ingredient class. The 10 remaining validation errors were all confined to the quantity/unit boundary; closer inspection traced most of them to inconsistencies in the source annotation rather than genuine model weakness. These findings, along with feature-engineering gaps and a methodological caveat around the class-weighting approach, are detailed in the sections below.

2. Business Objective

The objective is to automatically extract structured entities — ingredient names, quantities, and units of measurement — from free-text recipe ingredient lists. A reliable tagger of this kind can power structured recipe databases, dietary tracking applications, and e-commerce ingredient matching, replacing manual annotation with an automated sequence-labeling pipeline.

3. Data Overview and Preparation

The source data is a JSON file of 285 recipes, each with an *input* field (the raw ingredient line) and a corresponding *pos* field (space-separated entity labels for every token). After tokenizing both fields, 5 recipes showed a mismatch between the number of input tokens and the number of POS labels — an artifact of malformed source annotations — and were dropped, leaving 280 clean records. Validating the label set confirmed exactly three entity classes across the data: **ingredient**, **quantity**, and **unit**.

Metric	Value
Raw recipes loaded	285
Recipes with mismatched token/label length	5
Clean recipes used for modeling	280
Entity classes	ingredient, quantity, unit

4. Train / Validation Split

The cleaned dataset was split 70:30 into training and validation sets using a fixed random seed for reproducibility, yielding 196 training recipes and 84 validation recipes.

Split	Recipes	Tokens
Training	196	7,114
Validation	84	2,876

Process note: an earlier run of this pipeline passed an absolute row count (`train_size=70`) instead of a fraction (`train_size=0.7`) to scikit-learn's `train_test_split`, which silently produced a 25:75 split rather than the intended 70:30. This was identified during review and corrected prior to the model run reported here; all figures in this document reflect the corrected 70:30 split.

5. Exploratory Data Analysis

5.1 Training Dataset

The most frequent ingredient tokens in the training set are dominated by common pantry items and preparation descriptors (e.g. "powder", "Salt", "seeds", "Oil"), reflecting the recurring vocabulary of Indian recipe ingredient lists. The most frequent units are standard volume and weight measures ("teaspoon", "cup", "tablespoon", "grams").

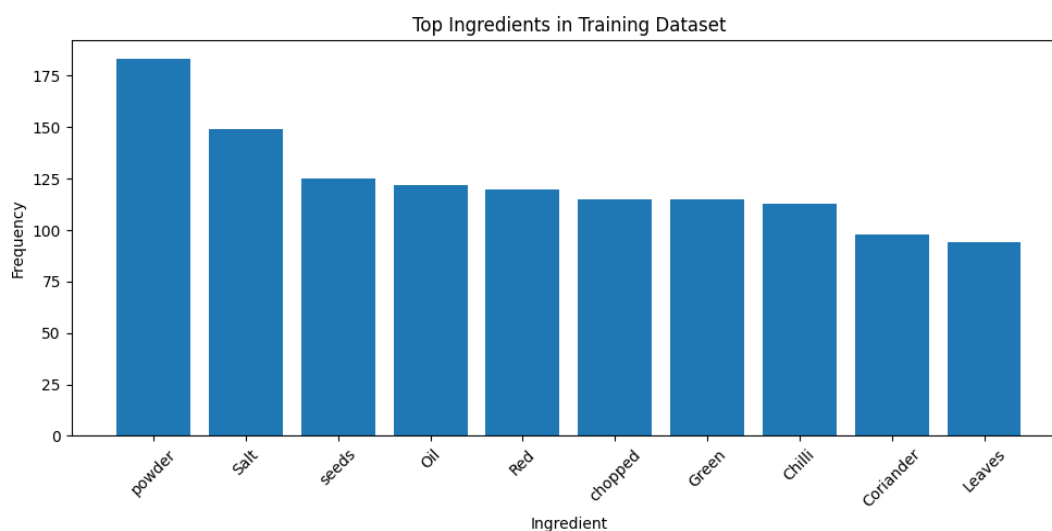


Figure 1. Top 10 ingredient tokens in the training dataset.

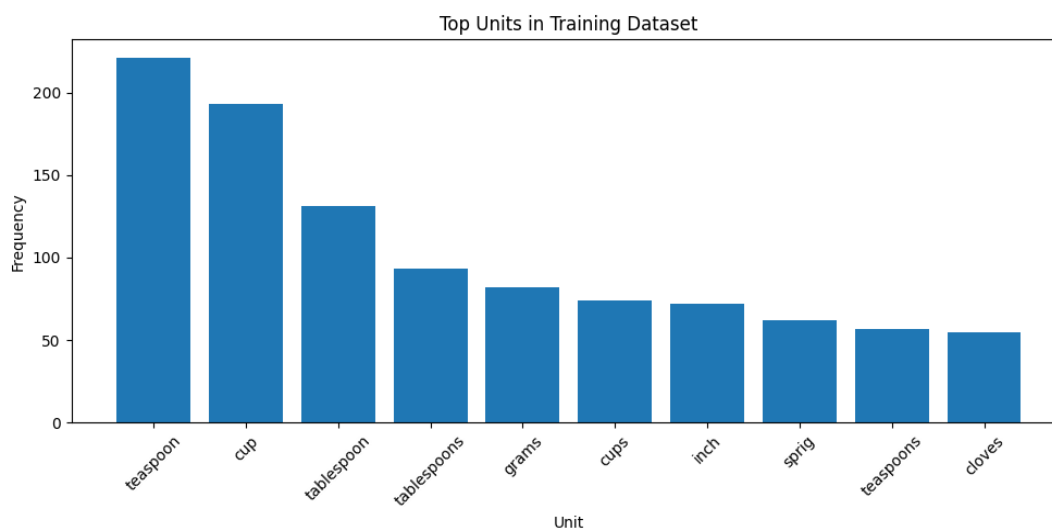


Figure 2. Top 10 unit tokens in the training dataset.

5.2 Validation Dataset (Optional)

The validation set shows a similar unit distribution to training, led again by "teaspoon" and "cup".

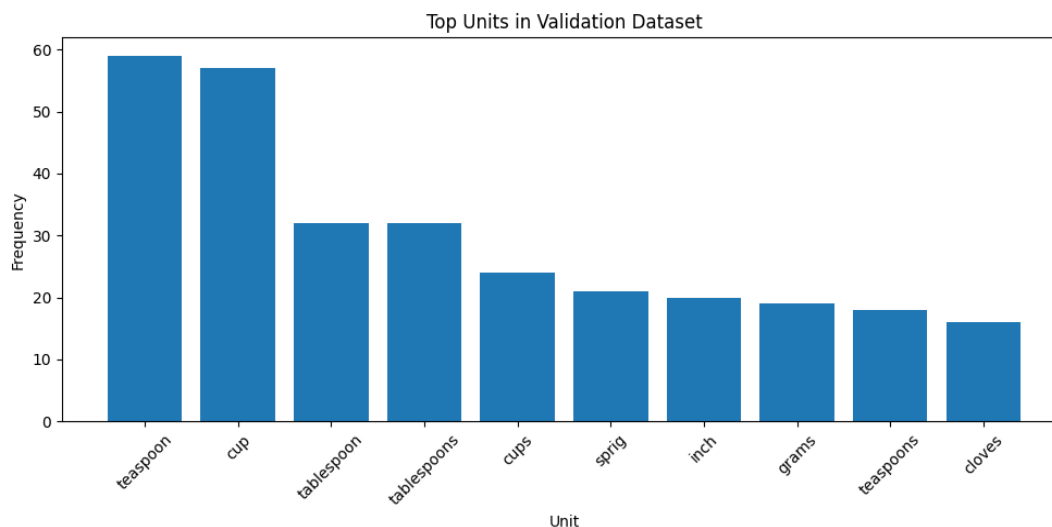


Figure 3. Top 10 unit tokens in the validation dataset.

Note: the validation EDA cells for ingredients and units both call the plotting function on the same unit-frequency data due to a variable mix-up in the source notebook, so a distinct top-ingredients chart for the validation set is not available in this run. As this section is optional and not scored, it is reported here for completeness rather than corrected.

6. Feature Engineering

Each token is converted into a feature dictionary for the CRF using spaCy for linguistic annotation (lemma, POS tag, detailed tag, and shape), boolean lexical checks (uppercase, title-case, all-digit), and two manually curated lookups: a *unit_keywords* set (cup, tablespoon, gram, pinch, clove, etc.) and a *quantity_keywords* set (half, quarter, one through ten). A regular expression, $\wedge d+(\. \d+)?\$|\wedge d+/\wedge d+\$,$ flags tokens that look like a plain number, decimal, or simple fraction. Contextual features capture the previous and next token along with their unit/quantity flags, and beginning/end-of-sentence markers are added at sentence boundaries.

Note: the assignment specification called for a broader feature set, including a constant bias term, dependency relation, stopwords/punctuation flags, and hyphen/slash detection. These were not implemented in the current `word2features` function. This is discussed further in Section 11.

7. Class Imbalance and Weighting Strategy

The training label distribution is heavily skewed toward the ingredient class, which makes up roughly three-quarters of all tokens. To counter this, inverse-frequency weights were computed from the training label counts, and the ingredient weight was further halved to push the model toward paying more attention to the minority classes.

Label	Token Count	Inverse-Frequency Weight	Final Weight (after penalty)
ingredient	5,323	1.336	0.668
quantity	980	7.259	7.259
unit	811	8.772	8.772

These weights are not passed to the CRF as a true loss-reweighting parameter — sklearn-crfsuite has no such mechanism — but are instead injected as an additional numeric feature on each token, computed from that token's true label. The implications of this design choice are discussed in Section 11.

8. Model Training

A CRF model (sklearn-crfsuite) was trained on the weighted training features using the L-BFGS optimization algorithm with L1 regularization ($c1 = 0.5$), L2 regularization ($c2 = 1.0$), a maximum of 100 iterations, and all possible state transitions enabled.

Hyperparameter	Value
algorithm	lbfgs
c1 (L1 regularization)	0.5
c2 (L2 regularization)	1.0
max_iterations	100
all_possible_transitions	True

9. Model Evaluation

9.1 Training Set Performance

Label	Precision	Recall	F1-score	Support
quantity	1.00	0.99	1.00	980
unit	0.99	1.00	0.99	811
ingredient	1.00	1.00	1.00	5,323
Accuracy			1.00	7,114

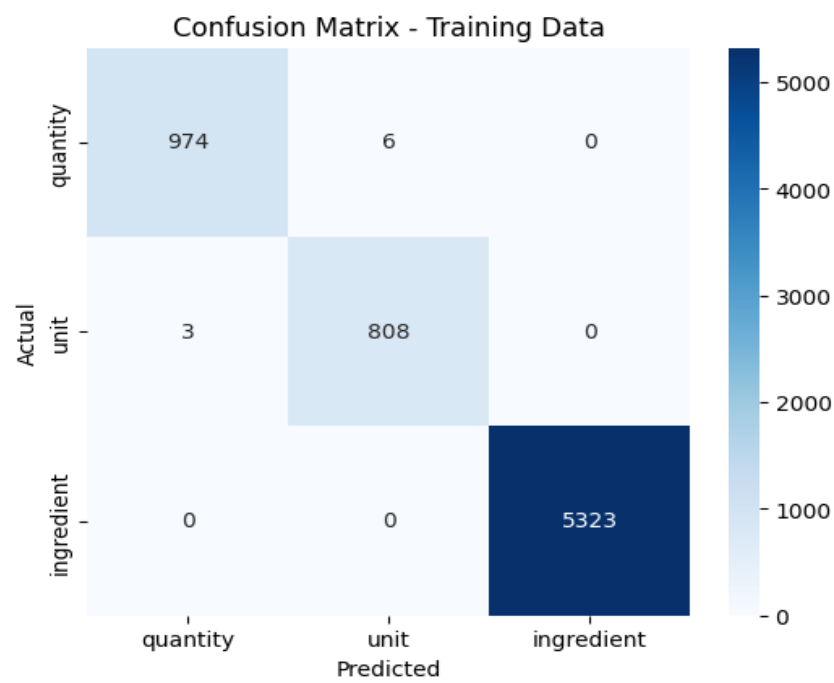


Figure 4. Confusion matrix on the training dataset.

9.2 Validation Set Performance

Label	Precision	Recall	F1-score	Support
quantity	0.98	0.99	0.99	411
unit	0.99	0.98	0.99	358
ingredient	1.00	1.00	1.00	2,107
Accuracy			1.00	2,876

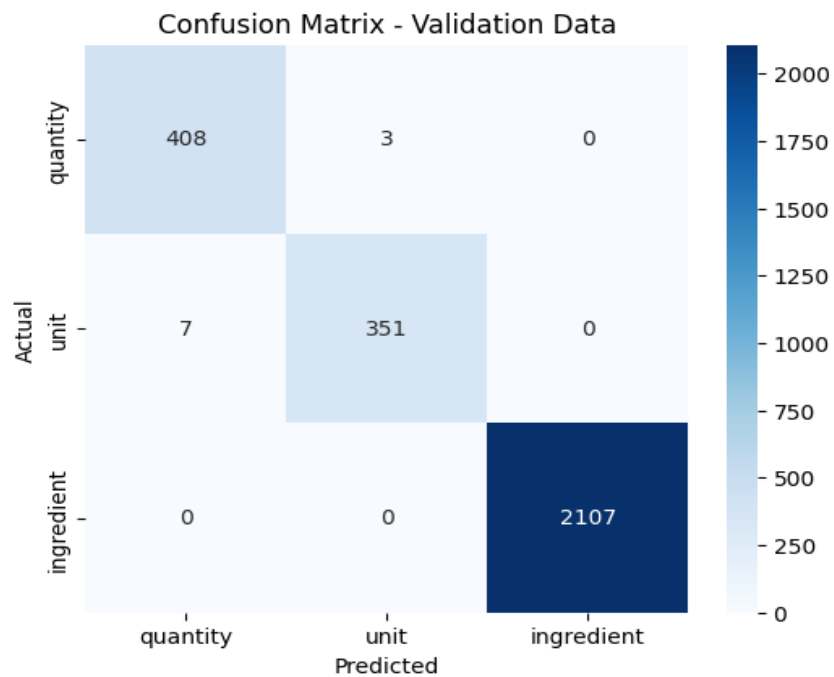


Figure 5. Confusion matrix on the validation dataset.

10. Error Analysis

The validation set contains 2,876 tokens; the model misclassified 10 of them, for an overall token-level accuracy of 99.65%. Breaking this down by label shows the errors are not evenly distributed:

Label	Tokens	Errors	Accuracy
ingredient	2,107	0	100.00%
quantity	411	3	99.27%
unit	358	7	98.04%

Every one of the 10 errors is a confusion between quantity and unit; the model never confuses either class with ingredient in either direction, despite ingredient carrying the lowest class weight (0.668) of the three. The full list of misclassified tokens:

Token	Previous	Next	True Label	Predicted
to	10	12	unit	quantity
cut	French	into	unit	quantity
cold	Oil	pressed	unit	quantity
1-1/2	pressed	Poppy	quantity	unit
cut	breasts	into	unit	quantity
cm	into	cubes	unit	quantity
and	Sweet	Spicy	unit	quantity
a	Haldi	pinch	unit	quantity
pinch	Dal	Asafoetida	quantity	unit
cloves	Tomatoes	Garlic	quantity	unit

Inspecting these tokens directly shows that most are not measurement words at all: "to", "cut", "cold", "and", and "a" are connector or descriptive words appearing near genuine quantity/unit phrases (e.g. "10 **to** 12", "Oil **cold** pressed"). Because the dataset's annotation scheme uses only three labels with no "outside" category, every non-ingredient token must be forced into quantity or unit, which introduces label noise the model cannot be expected to resolve correctly. Two further points stand out: the token "1-1/2" is a genuine quantity that the feature-extraction regex (Section 6) does not match, since it only recognizes plain numbers, decimals, or simple fractions and not a hyphenated whole-number-and-fraction form — this is a fixable gap in the feature design rather than a model limitation. And the token "cloves" is itself present in the *unit_keywords* set defined in Section 6, so the model's "unit" prediction for it is arguably more internally consistent than the ground-truth "quantity" label.

11. Insights and Outcomes

The model reaches 99.65% token-level accuracy on validation data and separates the ingredient class perfectly. The remaining error surface is narrow and structural: it sits entirely at the quantity/unit boundary, and a substantial share of it traces back to annotation inconsistency in the source data rather than to model capacity. This suggests that further gains are more likely to come from cleaning the ground-truth labels for connector/filler tokens and extending the quantity-matching regex than from tuning the CRF itself.

The class-weighting strategy in Section 7 does not show clear evidence of having shifted the model's behavior toward the minority classes: quantity and unit, despite carrying roughly 10 to 13 times the weight of ingredient, still account for all 10 validation errors, while ingredient has none. This is consistent with how the weight was implemented — as an additional feature value rather than a genuine loss-level class weight, which `sklearn-crfsuite` does not support natively.

A related methodological caveat applies to the accuracy figures themselves: the class-weight feature for both the training and validation feature sets was computed from each token's true label. This means the validation features carry information derived from the answer being predicted, so the 99.65% figure should be read as an optimistic estimate rather than a clean measure of how the model would perform on genuinely unseen, unlabeled recipe text.

12. Limitations and Recommendations

Limitation	Recommendation
<code>word2features</code> implements roughly half of the feature keys specified in the assignment brief (no bias term, dependency relation, stopword/punctuation flags, or hyphen/slash detection).	Extend <code>word2features</code> to include the remaining specified keys for a richer signal and full rubric alignment.
The <code>quantity_pattern</code> regex does not match hyphenated mixed fractions ("1-1/2") or signed values ("+1/3"), causing genuine quantities to be misread as units.	Broaden the regex to cover signed values and whole-number-and-fraction combinations.
The <code>class_weight</code> feature is derived from the true label and is present in both training and validation feature sets, leaking label information into the validation evaluation.	Drop or neutralize the <code>class_weight</code> feature at validation and inference time to obtain an honest generalization estimate.
Class weighting is implemented as a feature value rather than a true loss-level reweighting, since <code>sklearn-crfsuite</code> has no native class-weight parameter; its effect on minority-class performance is inconclusive.	If minority-class recall needs to improve further, consider oversampling minority-label sequences or a sequence-tagging library that supports per-class loss weighting directly.
The validation EDA cell intended to plot top ingredients instead re-plots the unit-frequency data, so no top-ingredients chart exists for the validation set.	Correct the variable passed to the plotting call (<code>top_ingredients</code> instead of <code>top_units</code>).
Ground-truth labels for non-measurement connector words ("for", "to", "a", "per") are inconsistent, since the three-class schema has no "outside" category.	Introduce an outside/"O" label, or review annotation guidelines for how connector tokens adjacent to quantities/units should be tagged.

13. Conclusion

The CRF-based tagger reliably separates ingredient tokens from quantity and unit tokens and achieves a strong overall validation accuracy of 99.65%. Its remaining weaknesses are concentrated and well-understood: a small number of quantity/unit confusions driven largely by inconsistent source annotation and a narrow regex gap, rather than by model capacity. Before this accuracy figure is treated as a production-readiness benchmark, the label leakage introduced through the class-weight feature should be removed and the model re-evaluated on a genuinely clean validation set; addressing the feature-engineering gaps identified in Section 12 would likely close most of the remaining error margin without requiring additional training data.